

Checklist: OCTMNIST 10-seed + Bảng tham số & chi phí

Hai việc dữ liệu cần hoàn tất TRƯỚC khi khóa bảng số để viết báo

Chào em. Ba giai đoạn em làm rất tốt. Trước khi ráp bảng kết quả vào bài báo, cần bổ sung 2 việc để bảng số vững và có thêm 'điểm bán' hiệu quả tham số: **(1) nâng OCTMNIST từ 5 → 10 seed** cho đồng nhất và để kiểm định thống kê có hiệu lực; **(2) lập bảng số tham số + chi phí** (đã kèm script tự đo). Làm xong 2 việc này là có thể bắt tay viết Results.

Vì sao phải nâng OCT lên 10 seed (đọc kỹ)

Hiện OCT chạy 5 seed **[0, 42, 100, 2023, 777]**. Với $n=5$, giá trị p nhỏ nhất mà kiểm định Wilcoxon có thể cho ra là **0,0625** — tức KHÔNG BAO GIỜ đạt được $p < 0,05$ dù kết quả có thật. BreastMNIST đã dùng 10 seed. Nâng OCT lên 10 seed vừa để kiểm định có hiệu lực, vừa để hai dataset đồng nhất khi trình bày.

1. Checklist chạy OCTMNIST 10 seed

Bước 1 — Sửa danh sách seed trong runner

Mở file `run_gd3.py` (ở thư mục gốc repo). Tìm khối **PART 2: OCTMNIST** (khoảng dòng 167–175). Sửa đúng 2 dòng:

```
# DÒNG ~167 (chỉ là dòng in, sửa cho khớp):
print(' [PART 2/2]: FULL OCTMNIST 3-TIER EXPERIMENT (5,000 SAMPLES, 10 SEEDS, L=1)')

# DÒNG ~175 – đây là dòng QUAN TRỌNG cần đổi:
# CŨ:
oct_seeds = [0, 42, 100, 2023, 777]
# MỚI (khớp đúng 10 seed của BreastMNIST):
oct_seeds = [0, 42, 100, 2023, 777, 999, 1234, 5678, 1111, 2222]
```

- ☐ Đã sửa `oct_seeds` thành đủ 10 seed, giống hệt `breast_seeds`.
- ☐ Kiểm tra xem repo có bản sao `GD3/run_gd3.py` không — nếu có và em muốn giữ đồng bộ, sửa luôn dòng tương ứng ở đó (nội dung giống hệt).

Bước 2 — Lường trước thời gian & tăng tốc

- Nâng 5 → 10 seed nghĩa là chạy GẤP ĐÔI số thí nghiệm phần trainable (mỗi seed × các mô hình trong ma trận 3 tầng). Phần mạch tĩnh dùng feature đã cache nên nhanh; phần **trainable** (backprop qua mạch) là phần tốn thời gian.
 - Nếu có GPU (Colab/Kaggle/qBraid miễn phí): bật để rút ngắn đáng kể. Nếu chạy CPU, nên chạy qua đêm.
 - KHÔNG tự merge kết quả 5 seed cũ với 5 seed mới bằng tay (dễ sai). Cứ chạy trọn 10 seed một lần cho sạch — file JSON sẽ được ghi đè đầy đủ.
- ☐ Đã quyết định chạy CPU hay GPU; nếu GPU thì đã kiểm tra PennyLane/PyTorch thấy thiết bị.

Bước 3 — Chạy lại thực nghiệm OCT

```
# Từ thư mục gốc repo:
python run_gd3.py

# (run_gd3.py chạy cả BreastMNIST + OCTMNIST. Nếu chỉ muốn chạy lại OCT
# để tiết kiệm thời gian, có thể tạm comment khối PART 1 BreastMNIST
# vì kết quả Breast 10 seed đã đúng rồi.)
```

- ☐ Chạy xong, có log in ra bảng 3 tầng cho OCTMNIST không lỗi.

Bước 4 — Kiểm tra (verify) kết quả mới

Mở file `results/full_trainable_octmnist.json` và xác nhận:

- ☐ `"seeds"` có đủ 10 phần tử; `"progress"` là `"60/60 completed"` (10 seed × 6 mô hình) thay vì 30/30.
- ☐ Trong `statistical_tests`, các `wilcoxon_p_value` giờ có thể xuống dưới 0,05 (không còn bị chặn ở 0,0625).
- ☐ Các hình trong `results/figures/gd3_octmnist_*.png` đã được vẽ lại (ngày sửa đổi mới).

Bước 5 — Cập nhật báo cáo & SỬA kết luận 'quán quân'

Đây là bước quan trọng nhất về mặt học thuật. Sau khi có số 10 seed:

- ☐ Cập nhật bảng OCT trong `GD3/BAO CAO GIAI DOAN 3.md` bằng số 10 seed mới (mean ± std + p-value).
- ☐ **Sửa lại phát biểu kết luận:** KHÔNG viết 'mạch trainable 3 trực là mạch lượng tử tốt nhất'. Phát biểu đúng theo số:
(a) 3 trực thắng đối thủ CÙNG HỌ 'Fixed Strongly'; (b) trên OCT, mạch TĨNH `random_L1` mới là mạch lượng tử mạnh nhất; (c) classical vẫn dẫn trên OCT.
- ☐ Thêm 1 dòng xếp hạng 'mạch lượng tử mạnh nhất theo từng metric' cho mỗi dataset.

2. Mẫu bảng SỐ THAM SỐ (đã điền sẵn)

Bảng dưới đã tính sẵn từ kiến trúc của em (script ở Phần 4 sẽ tự xác nhận lại). Ý tưởng trình bày: tách **feature-extractor** (phần tạo feature maps: conv cổ điển VS kernel lượng tử) khỏi **classifier head** (Linear 784→lớp, GIỐNG NHAU hai phía → đảm bảo công bằng).

BreastMNIST (2 lớp, L=2)

Mô hình	Feature-extractor (kernel+BN)	Classifier head	TỔNG (trainable)
Classical CNN	28 (conv 20 + BN 8)	1.570	1.598
Fixed Quanvolution (mạch tĩnh)	8 (kernel 0 + BN 8)	1.570	1.578
Trainable Basic (L=2)	16 (kernel 8 + BN 8)	1.570	1.586
Trainable Strongly 3-axis (L=2)	32 (kernel 24 + BN 8)	1.570	1.602

OCTMNIST (4 lớp, L=1)

Mô hình	Feature-extractor (kernel+BN)	Classifier head	TỔNG (trainable)
Classical CNN	28 (conv 20 + BN 8)	3.140	3.168
Fixed Quanvolution (mạch tĩnh)	8 (kernel 0 + BN 8)	3.140	3.148
Trainable Basic (L=1)	12 (kernel 4 + BN 8)	3.140	3.152
Trainable Strongly 3-axis (L=1)	20 (kernel 12 + BN 8)	3.140	3.160

Luận điểm rút ra từ bảng tham số (dùng cho Discussion)

- Phần classifier head (Linear 784→lớp) chiếm gần hết tham số và GIỐNG NHAU hai phía → chênh lệch tổng là không đáng kể, nhưng đó chính là điều khiến so sánh công bằng.
- Bộ tạo đặc trưng lượng tử dùng 0 (tĩnh) đến 24 (trainable) tham số — tương đương hoặc ÍT HƠN conv cổ điển (20). Đặc biệt: mạch TĨNH đạt hiệu năng cạnh tranh với 0 tham số học ở tầng đặc trưng → luận điểm 'quantum inductive bias' + hiệu quả tham số.
- Kết hợp với độ lệch chuẩn nhỏ hơn ~3× (đã có ở GĐ2–3): thông điệp trung thực là 'không phải accuracy cao hơn, mà ỔN ĐỊNH hơn với chi phí tham số tầng đặc trưng tương đương/ít hơn'.

3. Mẫu bảng CHI PHÍ / THỜI GIAN (điền từ số đo thực)

Bảng này lấy số THỰC TẾ trên máy em: cột thời gian huấn luyện đọc từ log khi chạy `run_gd3.py`; cột độ trễ suy luận & bộ nhớ lấy từ script ở Phần 4. Ghi rõ cấu hình máy (CPU/GPU, RAM).

Cấu hình máy đo: _____ (VD: CPU Intel i5-... / GPU Colab T4 / RAM ...GB)

Mô hình	Precompute feature (1 lần, phút)	Train time / seed (phút)	Tổng wall-clock 10 seed (phút)	Đỉnh bộ nhớ (GB)	Suy luận (ms/ảnh)
Classical CNN	— (không có)	_____	_____	_____	_____
Fixed Quanvolution	_____	_____	_____	_____	_____
Trainable Basic	— (in-loop)	_____	_____	_____	_____
Trainable Strongly 3-axis	— (in-loop)	_____	_____	_____	_____

Gợi ý cách lấy số: (1) thời gian huấn luyện: script GD3 đã in tiến độ theo thời gian — ghi lại; nếu chưa, thêm `time.perf_counter()` quanh vòng lặp seed. (2) độ trễ suy luận + kích thước mô hình: chạy script `measure_params_cost.py` ở Phần 4. (3) bộ nhớ đỉnh: CPU dùng `psutil`; GPU dùng `torch.cuda.max_memory_allocated()`.

Điểm cần trung thực trong bảng chi phí

Quanvolution (nhất là trainable) CHẬM hơn classical nhiều lần — hãy báo cáo thẳng con số. Đây không phải điểm yếu chí mạng: bài báo định vị 'đánh đổi chi phí lấy độ ổn định + inductive bias trên dữ liệu ít mẫu', và mở hướng tăng tốc (GPU/cuQuantum, FPGA) ở phần Future Work.

4. Dùng script tự đo (`measure_params_cost.py`)

Kèm theo tài liệu này là file `measure_params_cost.py`. Nó tự đếm tham số (khớp bảng Phần 2) và đo độ trễ suy luận, rồi in ra bảng Markdown dán thẳng vào bài.

```
# 1. Đặt file vào THƯ MỤC GỐC repo (cùng cấp run_gd3.py)
# 2. Chạy:
python measure_params_cost.py
```

- Script ưu tiên import model thật từ `src/`; nếu import lỗi, nó tự dùng bản sao inline khớp kiến trúc (vẫn ra đúng số tham số).
- Phần đo độ trễ quanvolution cần PennyLane (để gọi mạch thật); nếu thiếu, script vẫn chạy và bỏ qua phần đó (in cảnh báo).
- Cột 'Train time / seed' và 'Đỉnh bộ nhớ' KHÔNG do script này đo — lấy từ log `run_gd3.py` như hướng dẫn Phần 3.

5. Tiêu chí HOÀN THÀNH việc bổ sung này

- ☐ `results/full_trainable_octmnist.json` có 10 seed (60/60 completed).
- ☐ Bảng OCT trong báo cáo GD3 đã cập nhật theo 10 seed; kết luận 'quán quân' đã viết lại cho đúng số.
- ☐ Bảng tham số (Phần 2) đã được script xác nhận trùng khớp.
- ☐ Bảng chi phí (Phần 3) đã điền đủ số đo thực + ghi cấu hình máy.
- ☐ Đã commit/push toàn bộ (JSON + figures + báo cáo + script) lên GitHub.

Làm xong 5 mục trên là dữ liệu đủ 'khóa' để bắt đầu viết Results. Có gì vướng khi chạy, em chụp log gửi thầy nhé.

Hướng dẫn dựa trên khảo sát trực tiếp mã nguồn repo `NamIsStudyingCE/Quanvolution`. Số tham số tính từ kiến trúc thực tế (`SymmetricalMinimumCNN`, `QuanvolutionClassifier`, `TrainableQuanvLayer`).